arXiv:2605.17909v2 [cs.AI] 31 May 2026

Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems

Riddhi Mohan Sharma, *Senior Member, IEEE*
IAM Architect & Non-Human Identity Governance
Enterprise Identity, M&A Integration, Regulated Environments
Email: riddhimohan@ieee.org

***Abstract*—As autonomous agentic systems scale across regulated critical infrastructures, the lack of mechanistic, hardware-rooted enforcement for high-frequency policy updates presents a fundamental safety gap. We present Ethical Hyper-Velocity (EHV), a governance-aware runtime enforcement architecture for agentic systems that combines Grammar-Constrained Decoding (GCD) for inline policy-constrained token generation, Causal Graph CRDT-based policy synchronization with vector-clock ordering, hardware-attested execution in Trusted Execution Environments (TEEs), and OSCAL-formatted machine-readable audit logging. Unlike retrospective auditing frameworks (ISO/IEC 42001, NIST AI RMF) that introduce 14–30 day policy latencies, EHV relocates the Policy Enforcement Point (PEP) into the inference pipeline via a Governance-Aware Just-In-Time (JIT) Compiler. Under explicitly stated assumptions, the architecture reduces enforcement latency, improves traceability, and supports formal verification of safety invariants in a bounded model. We demonstrate via TLA+ model checking that non-compliant agentic actions were unreachable in the verified bounded operating state space (1,738 states generated, 324 distinct, depth 8, zero violations). Under these conditions, $\mathcal{O}(1)$ runtime enforcement reduces the traditional trade-off between deployment velocity and governance integrity, targeting Governance Latency from $\mathcal{O}$(days) toward $\mathcal{O}(1)$. EHV's differentiating contribution is the integration of GCD, Causal CRDT, TEE attestation caching, and bounded formal verification into a single, hardware-rooted enforcement architecture—a combination not achieved by any contemporaneous system. The architecture is demonstrated through a pediatric oncology dosage use case, with applicability to regulated critical infrastructures including healthcare, financial compliance, and critical infrastructure control.**

***Index Terms*—AI Governance, Agentic Systems, Zero Trust Architecture, Grammar-Constrained Decoding, Formal Verification, TLA+, Trusted Execution Environments, CRDT, Healthcare AI, Governance Latency, SPIFFE, OSCAL**

I. Introduction

Current AI governance relies primarily on retrospective auditing and manual compliance gates. As autonomous agents—systems capable of multi-step reasoning and real-world action without human intervention—proliferate in regulated domains, this creates a *Governance Bottleneck*: model execution velocity exceeds oversight velocity by orders of magnitude.

Existing standards address the risk lifecycle (NIST AI RMF [1]) and management systems (ISO/IEC 42001 [2]), but they lack a *mechanistic enforcement model* at the execution layer. We argue that for autonomous agents in high-stakes regulated environments, governance must move from a procedural gate to a formally constrained architectural design.

We introduce **Ethical Hyper-Velocity (EHV)**, a framework for the **runtime enforcement of AI governance policies**. EHV provides an architecture for hardware-rooted, real-time policy enforcement that supports safety invariants under high-frequency policy updates and distributed network conditions. By compiling governance into the inference stack via Grammar-Constrained Decoding, we transform policy from an external friction point into an architecturally constrained system property.

A. Contributions

This paper makes the following contributions:

1) **The Governance Latency Problem**: We formalize the metric $GL = t_{\mathrm{e}} - t_{\mathrm{d}}$ and demonstrate its measurable implications in regulated autonomous systems, including a 168M-action unsafe-action exposure calculation for a healthcare deployment.
2) **The Identity-Action Perimeter**: We extend NIST SP 800-207 ZTA [3] from identity verification to agentic action verification, closing the "trusted identity, untrusted action" gap via per-action authorization binding.
3) **Governance-Aware JIT Compiler with GCD**: We present an architecture that elevates Grammar-Constrained Decoding to the primary enforcement path, relocating the PEP into the token-generation layer, backed by Causal CRDT policy synchronization and TEE-anchored epoch caching.
4) **Bounded Formal Safety Verification**: We provide TLA+ specifications demonstrating that non-compliant actions were unreachable in the verified bounded model under all explored state-space interleavings at depth 8.
5) **Structured Threat Model**: We enumerate the attack surface and failure modes specific to governance-

compiled agentic systems, including 2026-class hardware vulnerabilities.

6) **Regulatory Alignment**: We map EHV controls to NIST AI RMF, EU AI Act Article 12, NIST SP 800-207, and FDA PCCP requirements.

II. Related Work

A. AI Governance Frameworks

The NIST AI Risk Management Framework [1] provides a lifecycle-oriented taxonomy (Map, Measure, Manage, Govern) but offers no enforcement mechanism; compliance is assessed retrospectively. ISO/IEC 42001 [2] establishes a management system for AI but inherits the PDCA audit cycle, yielding $GL \geq 14$ days. The EU AI Act [4] mandates risk classification and conformity assessment for high-risk systems under Annex III, and requires automatic logging under Article 12. None of these frameworks address real-time, pre-execution constraint enforcement for autonomous agents executing at machine speed.

B. Zero Trust Architecture

NIST SP 800-207 [3] defines Zero Trust Architecture as continuous verification of the *subject* (identity) requesting access. In agentic contexts, this is necessary but insufficient: a cryptographically authenticated "Physician Twin" with valid credentials can still execute an action that violates a policy updated seconds ago. ZTA verifies *who*; EHV verifies *what*.

C. Formal Methods in Safety-Critical Systems

TLA+ and the TLC model checker [5] have been applied to distributed systems verification at Amazon Web Services [7] and Microsoft Azure. Formal verification of AI safety constraints remains nascent. Amodei et al. [8] enumerate concrete AI safety problems but do not propose mechanistic enforcement. Constitutional AI [9] embeds behavioral constraints in training but provides no runtime guarantee. EHV bridges this gap by applying formal methods to the *governance enforcement layer* rather than the model itself.

D. Trusted Execution Environments

Intel SGX [10], AMD SEV-SNP [11], and ARM TrustZone provide hardware-rooted isolation for sensitive computation. Remote attestation protocols verify enclave integrity but introduce 200ms+ latency per attestation round-trip. EHV's Epoch-based Attestation Caching amortizes this cost to $\mathcal{O}(1)$ per inference call within an epoch.

E. CRDTs for Distributed State

Conflict-free Replicated Data Types [12] guarantee eventual consistency without coordination. Join-Semilattice structures ensure monotonic policy convergence. EHV leverages this property to propagate safety constraints across partitioned networks without a central policy bottleneck. The v2 architecture upgrades from Last-Write-Wins timestamps to Causal Graph CRDTs with vector clocks [6], removing NTP dependency and physical clock injection attacks.

F. Runtime Guardrail Systems

NVIDIA NeMo Guardrails [15] and Guardrails AI [16] provide runtime constraint enforcement for LLM outputs via programmable rules. These operate as software-layer filters between the model and the user. EHV differs in three fundamental dimensions: (1) *formal verification*—runtime guardrail systems offer no proof that unsafe outputs are unreachable; (2) *hardware-rooted enforcement*—EHV's PEP executes within a TEE, making bypass via process-level attacks significantly harder; (3) *grammar-constrained token generation*—EHV masks invalid tokens at the logit level before sampling, rather than filtering complete output strings.

G. Grammar-Constrained Decoding Foundations

Grammar-Constrained Decoding (GCD) compiles a context-free grammar into a DFA, intersects the active generation prefix with allowed DFA transitions, and masks logits for disallowed tokens to $-\infty$ before softmax. This approach is implemented in XGrammar [18], Outlines [19], LMQL, and vLLM's LogitsProcessor API. EHV adapts this technique specifically for policy enforcement: the grammar encodes the complete set of permissible agentic actions as a finite automaton, converting policy compliance into a token-generation constraint.

H. Emerging Agentic Runtime Security Systems (2025–2026)

Several contemporaneous systems address runtime governance for agentic AI:

MI9 [25] provides telemetry, continuous authorization, and containment for agentic AI systems. It is the most directly comparable system in scope—action-level governance at the application layer—but operates without hardware TEE backing.

AgentSpec [26] offers customizable runtime enforcement for LLM agents using declarative policy specifications. Similar in motivation but lacks formal verification and hardware attestation.

Aegis [27] mediates model outputs through a trusted decision layer at the application tier. No hardware-rooted enforcement or formal model checking.

SAFi [28] is an open-source runtime governance engine with deterministic gating and audit logging. It provides no formal verification.

AgenTEE [29] is the closest architectural parallel: it isolates agent runtimes in TEEs using Arm CCA. However, AgenTEE does not integrate GCD for token-level enforcement, CRDT-based policy synchronization, or bounded formal verification.

AutoTEE [30] automates porting of agent functions into TEEs using LLM assistance, and could serve as an implementation complement to EHV.

EHV's differentiating contribution: no existing system combines GCD (token-level grammar enforcement), Causal CRDT (distributed policy synchronization), TEE attestation caching, and bounded formal verification (TLA+/TLC) in a single architecture. The contribution is not the invention of any individual component—GCD, CRDTs, TEEs, and model checking are

independently well-established—but the *co-design protocol* that orchestrates these mechanisms into a closed enforcement loop: GCD alone does not guarantee policy freshness (CRDT provides this), TEE alone does not constrain token generation (GCD provides this), CRDT alone does not prevent policy application on a compromised host (TEE attestation provides this), and formal verification alone does not enforce at runtime (GCD+TEE provides this). This four-way co-design creates emergent enforcement properties that no individual component achieves.

I. NIST CAISI and Agentic Identity Standards

The NIST Center for AI Standards and Innovation (CAISI) launched the AI Agent Standards Initiative in February 2026 [22], addressing interoperability, authentication, authorization, and identity management for AI agents. The companion NCCoE concept paper on "Accelerating the Adoption of Software and AI Agent Identity and Authorization" [23] proposes adapting existing identity protocols (OAuth 2.0/2.1, OIDC, SPIFFE/SPIRE) for non-human agent workloads. EHV's workload identity model (Section IV-E) aligns with and supports the goals of these emerging standards.

The Model Context Protocol (MCP), donated to the Linux Foundation's Agentic AI Foundation in December 2025 [24], is emerging as a widely adopted open protocol for agent-to-tool communication. EHV's action authorization layer is compatible with MCP-structured tool calls, enabling governance enforcement at the protocol boundary.

III. Problem Formulation: Governance Latency

Definition 1 (Governance Latency)**.** *For a policy decision event at time t_{d} and its enforcement at time t_{e}, Governance Latency is defined as:*

$$GL = t_{\mathrm{e}} - t_{\mathrm{d}}, \quad GL \in [0, \infty) \tag{1}$$

In traditional frameworks, GL spans 14–30 days due to manual review cycles. During this interval, an autonomous agent may execute N actions under a stale policy state:

$$N_{\text{unsafe}} = \lambda \cdot GL \tag{2}$$

where λ is the agent's action throughput (actions/second). For illustrative healthcare parameters—$I = 5{,}000$ Physician Twin instances, $R = 100$ recommendations/hour—a 14-day GL yields:

$$\begin{aligned} N_{\text{unsafe}} &= I \cdot R \cdot 24 \cdot GL_{\text{days}} \\ &= 5{,}000 \times 100 \times 24 \times 14 = 168{,}000{,}000 \end{aligned} \tag{3}$$

This 168M-action exposure under a stale policy state establishes the *magnitude* of the Governance Latency problem. EHV's objective is to reduce GL toward a constant bounded by TEE attestation overhead:

$$GL_{\text{EHV}} = \mathcal{O}(1), \quad GL_{\text{EHV}} < 1\text{ms (target)} \tag{4}$$

Note: The < 1ms target is an architectural design objective derived from TEE LogitsProcessor overhead estimates. Empirical validation on production AMD SEV-SNP hardware is identified as primary future work (Section IX).

CAP Theorem Positioning: EHV is a CP system under the CAP theorem: it prioritizes Consistency (policy enforcement correctness) and Partition Tolerance (CRDT convergence) over Availability. When a network partition exceeds the epoch boundary $|E_k|$, EHV sacrifices availability by transitioning to a fail-closed halt state (Section VI). In safety-critical regulated environments, halting an agent is strictly preferable to executing under a stale or unverified policy. The $\mathcal{O}(1)$ latency target refers to *enforcement latency*—the per-token GCD check during steady-state within a valid epoch—not end-to-end system latency including policy propagation, which is $\mathcal{O}(\text{network})$.

IV. System Architecture

The EHV architecture consists of four technical pillars: a Causal Graph CRDT Policy Compiler, Epoch-based Attestation Caching, a Grammar-Constrained Decoding PEP, and a SPIFFE/SPIRE Workload Identity layer.

A. Pillar 1: Causal Graph CRDT Policy Compiler

Policies are ingested as monotonic updates in a Causal Graph CRDT using vector clock ordering. Each policy update p_i carries a vector timestamp $\vec{v}_i$. The merge function guarantees:

$$\forall p_i, p_j : \vec{v}_i \succ \vec{v}_j \implies \text{merge}(p_i, p_j) = p_i \tag{5}$$

where $\succ$ denotes causal dominance under the vector clock partial order. This ensures that all agent nodes eventually converge on the most recent safety constraints without coordination overhead, and without dependence on physical clock synchronization. The Global Ethical State S_G is defined as:

$$S_G = \bigsqcup_{i=1}^{n} S_i \tag{6}$$

where $\bigsqcup$ denotes the least upper bound in the semilattice and S_i is the local state at node i.

Clock Model. The v2 specification uses vector clocks (Lamport causal ordering [6]) in place of the v1 LWW physical timestamps. This removes the dependency on NTP clock synchronization, preventing malicious backdated policy injection (Threat T7, Section VI-B). In production deployments, S_G is represented as a directed acyclic graph (DAG) of cryptographically signed policy mutations resolved via vector clocks, removing all physical clock dependencies.

Policy Merkle Root. Each epoch's constraint set is committed as a Merkle root $H_{\mathrm{p}} = \text{SHA-256}(S_G)$ enabling $\mathcal{O}(1)$ integrity verification and GBOM binding (Section VIII-B).

B. Pillar 2: Epoch-based Attestation Caching

Remote hardware attestation (Intel TDX, AMD SEV-SNP) incurs 200ms+ latency per round-trip. EHV introduces *Policy Epochs*: the TEE validates the policy hash H_{p} once per epoch E_k. Within an epoch, the enforcement check reduces to:

$$\text{Verify}(a) = \begin{cases} \mathcal{O}(1), & \text{if } H_{\mathrm{p}}^{\text{local}} = H_{\mathrm{p}}^{\text{epoch}}, \\ \text{Re-attest}, & \text{otherwise.} \end{cases} \tag{7}$$

Epoch duration $|E_k|$ is configurable per domain. In the healthcare vertical, $|E_k| = 60$s balances freshness against attestation cost.

C. Pillar 3: Grammar-Constrained Decoding PEP

v2 Primary Enforcement Path. The v2 architecture elevates Grammar-Constrained Decoding (GCD) to the primary Policy Enforcement Point, replacing the v1 ASEL-based post-generation filtering. The PEP is relocated from an external gateway to the token-generation layer of the inference pipeline.

The governance policy Π is compiled into a Deterministic Finite Automaton (DFA) $\mathcal{A}_\Pi$ representing the complete set of permissible action sequences. At each generation step t, before softmax sampling, the GCD engine:

1) Computes the current DFA state q_t from the generated prefix.
2) Queries the set of valid transitions $\Sigma_t = \mathcal{A}_\Pi.\text{allowed}(q_t)$.
3) Sets logit scores for all disallowed tokens to $-\infty$:

$$L'_t[k] = \begin{cases} L_t[k], & \text{if } k \in \Sigma_t, \\ -\infty, & \text{otherwise.} \end{cases} \tag{8}$$

This ensures that the model's output distribution is structurally restricted to policy-compliant action tokens at every generation step. The constraint is architectural, not probabilistic: no token outside Σ_t can be sampled regardless of its logit score.

Double-Buffered DFA Update Protocol. When a CRDT policy convergence event produces an updated policy Π' during active token generation, the system does *not* stall the inference pipeline. Instead, EHV employs a double-buffered recompilation strategy: the updated CFG is compiled into a new DFA $\mathcal{A}_{\Pi'}$ in a background thread within the TEE. To prevent mid-inference undefined states or execution panics, EHV introduces *Prefix Alignment Validation*. The background compilation engine evaluates the current active DFA state q_t against the state space of the incoming automaton $\mathcal{A}_{\Pi'}$. The atomic swap of the active DFA pointer at step t is governed by:

$$\mathcal{A}_{\text{active}} \leftarrow \begin{cases} \mathcal{A}_{\Pi'} & \text{if } q_t \in \mathcal{A}_{\Pi'} \text{ and } t \in \mathcal{B}, \\ \mathcal{A}_{\text{active}} & \text{otherwise.} \end{cases} \tag{9}$$

where $\mathcal{B}$ denotes the set of transaction boundaries (e.g., between discrete action outputs). If $q_t \notin \mathcal{A}_{\Pi'}$, the atomic swap is deferred, and a local execution epoch freeze is maintained until the current transactional boundary completes. This ensures that (a) no generation stall occurs mid-sequence, (b) DFA consistency is maintained within a single action, and (c) policy freshness converges to the next action boundary rather than the next epoch boundary for already-received updates. This state variable has been formally modeled in the TLA+ specification, verifying safety under arbitrary concurrent interleavings.

Grammar Complexity Bound. The DFA state space grows with policy complexity. For enterprise deployments with n independent policy constraints, DFA intersection can produce $\mathcal{O}\left(\prod_i |q_i|\right)$ states in the worst case. The current architecture assumes domain-specific schemas of *bounded cardinality*—healthcare clinical action vocabularies typically yield DFAs with 10^2–10^4 states, well within TEE encrypted memory budgets. For deployments exceeding this range, hierarchical grammar decomposition (compiling independent constraint subsets into separate DFAs applied sequentially) is the recommended scaling strategy. Characterizing the performance cliff under expanding grammar profiles within TEE encrypted memory constraints is identified as primary future benchmarking work (Section IX).

Scope Boundary. GCD enforces *syntactic* compliance: the generated token sequence conforms to the compiled grammar. Semantic compliance (whether a syntactically valid action achieves a harmful goal via indirect means) remains outside the GCD enforcement boundary. A syntactically compliant but semanticaly harmful action will be executed and signed into the GBOM audit trail with a clean compliance record; the hardware-rooted enforcement secures the pipeline mechanics, not the semantic payload. Defense in depth via the ESCALATE path and external semantic analysis layers is required for high-stakes action categories. Grammar correctness is a trust dependency: if the policy CFG fails to capture a relevant constraint, GCD will not enforce it. The grammar specification process is identified as a primary engineering discipline for EHV deployments.

Before an agent emits an action a, the EHV JIT Compiler evaluates:

$$G(a, C) \in \{\texttt{PERMIT}, \texttt{DENY}, \texttt{ESCALATE}\} \tag{10}$$

where C is the current constraint set derived from S_G. `DENY` routes to a Safe Halt State. `ESCALATE` triggers a human-in-the-loop override for non-binary clinical judgment requiring policy interpretation beyond the compiled grammar.

D. Action Schema Extraction Layer (Compatibility Mode)

(ASEL is retained for backward compatibility with v1.0 deployments lacking GCD support. In v2, ASEL is not in the primary enforcement critical path.)

The PEP can evaluate structured action representations parsed from model outputs by a pre-PEP Action Schema Extraction Layer (ASEL). ASEL maps unstructured model output into typed action tuples $(\texttt{action_type}, \texttt{parameters}, \texttt{context})$. For example, the output "administer 1.5 mg/m^2 Vincristine IV" maps to

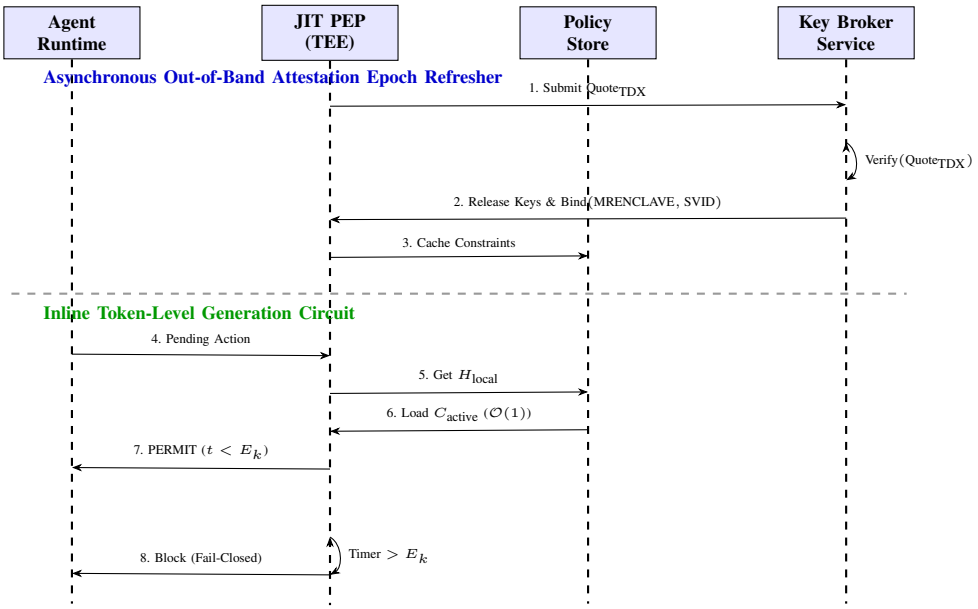


Fig. 1. The Core Decision Lifecycle. The JIT PEP restricts remote attestation to periodic epoch boundaries. If a network partition exceeds the epoch duration, the system structurally fails closed.

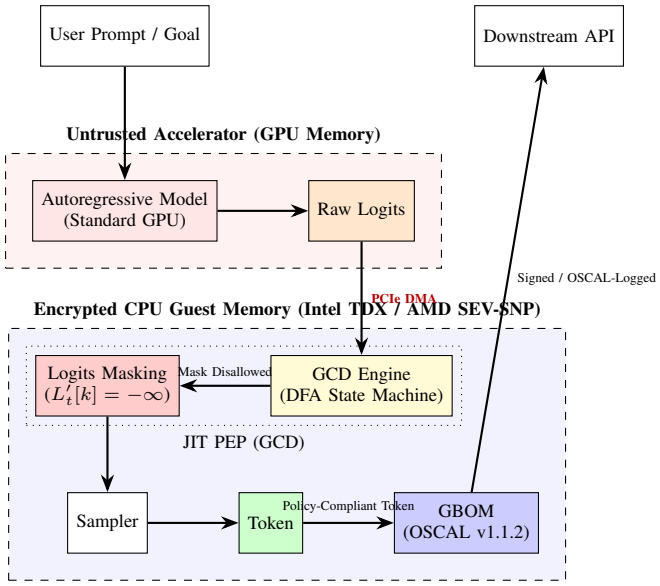


Fig. 2. System Architecture. The Autoregressive Model runs on an untrusted GPU accelerator, transferring Raw Logits via PCIe DMA to the JIT PEP executing inside encrypted CPU guest memory (TEE). The JIT PEP implements Grammar-Constrained Decoding, masking invalid token scores to $-\infty$ before sampling. Policy-compliant tokens are then logged to the OSCAL-formatted GBOM.

$(\texttt{DOSAGE}, \{\texttt{drug} : \texttt{Vincristine}, \texttt{dose} : 1.5, \texttt{unit} : \texttt{mg/m}^2\}, \texttt{IV})$.

ASEL fidelity is domain-specific and is *not* formally verified within the current EHV specification. The safety invariant I_g holds conditional on correct ASEL extraction. Migration to full GCD enforcement (Pillar 3) removes this unverified dependency.

E. Pillar 4: SPIFFE/SPIRE Workload Identity

In agentic deployments, each agent instance is a Non-Human Identity (NHI) that must be authenticated before action authorization can be applied. EHV integrates the SPIFFE (Secure Production Identity Framework For Everyone) standard and SPIRE runtime [20] to issue ephemeral, session-bound X.509 SVID credentials to each agent workload.

The identity model binds:

- **Workload identity**: SPIFFE SVID attesting agent instance provenance.
- **Action credential**: RFC 8693 Token Exchange [21]—scoped, short-lived OAuth 2.0 token authorizing a specific action class per epoch.

- **Attestation binding**: The TEE enclave measurement (MR) is bound to the action token, ensuring that credentials issued to a valid enclave cannot be replayed by a compromised host process.

This design supports the goals articulated in the NIST NCCoE concept paper on AI agent identity and authorization [23]: separating the workload identity lifecycle from the policy authorization lifecycle, enabling fine-grained per-action governance.

F. Separation of Duties and Fault Isolation

EHV separates three functional roles across distinct trust boundaries, aligning with NIST SP 800-53 AC-5 (Separation of Duties) [14]:

- **Policy Administration Point (PAP)**: External to the TEE. Human policy authors and automated CI/CD pipelines author, cryptographically sign, and publish governance policies. Signing keys are held in external Hardware Security Modules (HSMs) or key management services, never inside the TEE enclave.
- **Policy Decision/Enforcement Point (PDP/PEP)**: Inside the TEE. The JIT PEP verifies policy signatures, merges updates via the CRDT store, and enforces the compiled DFA. The PEP cannot author, modify, or override policies.
- **Audit Verification Point (AVP)**: External to the TEE. The TEE emits cryptographically signed GBOM records (Appendix C). Compliance verification, audit aggregation, and regulatory reporting are performed by independent external systems consuming these signed records.

This separation ensures that a compromised TEE cannot both enforce and audit its own decisions: a PEP breach results in fail-closed behavior (no actions permitted), not fraudulent policy injection or audit suppression, because the PAP signing keys and AVP verification logic reside outside the TEE trust boundary.

Intra-TEE TCB Considerations. Within the TEE guest memory, the JIT PEP process consolidates several functions: policy signature verification, CRDT lattice merging, vector clock state maintenance, CFG-to-DFA compilation, and logit masking execution. This consolidation expands the intra-TEE Trusted Computing Base (TCB). If an attacker achieves code execution inside the guest environment (e.g., via a microarchitectural exploit such as StackWarp), both policy convergence logic and token filtering are compromised simultaneously. EHV mitigates this risk through three controls: (1) the TEE guest image is a minimal, statically linked binary with no dynamic library loading, reducing the attack surface; (2) the PAP signing keys remain external, so a compromised PEP cannot inject fraudulent policies; and (3) the AVP independently verifies GBOM records, so audit integrity is maintained even if enforcement is compromised. Further TCB decomposition—isolating the CRDT state machine from the DFA execution engine into separate micro-enclaves—is identified as a hardening measure for future production deployments.

V. Formal Verification (TLA+)

We specify the EHV system in TLA+ with five state variables:

- `PolicySet`: The current CRDT-merged policy state.
- `AgentAction`: The pending action under evaluation.
- `NetworkState`: $\in \{\texttt{CONNECTED}, \texttt{PARTITIONED}\}$.
- `EnforcementStatus`: $\in \{\texttt{PERMIT}, \texttt{DENY}, \texttt{ESCALATE}\}$.
- `DfaState`: $\in \mathcal{A}_\Pi$.States representing the active DFA state q_t tracking token-level prefix alignment.

A. Safety Invariant

$$I_g \triangleq \forall a \in \mathit{AgentAction} : \neg\text{Valid}(a, \mathit{PolicySet}) \implies \mathit{Status} = \texttt{DENY} \quad (11)$$

This states that no invalid action can reach a `PERMIT` state regardless of the system's execution path.

B. Liveness Property

$$L \triangleq \forall p \in \mathit{PolicyUpdate} : \Diamond(\mathit{PolicySet}' = \text{merge}(\mathit{PolicySet}, p)) \quad (12)$$

Every policy update eventually propagates to all nodes (guaranteed by CRDT convergence).

C. Model Checking Results

Using the TLC Model Checker (v2026.05.04) with 10 parallel workers, covering all interleavings of asynchronous policy updates, network partitions, and concurrent agent actions:

- **States generated**: 1,738
- **Distinct states**: 324
- **State graph depth**: 8
- **Safety violations**: 0
- **Deadlocks**: 0
- **Temporal property violations**: 0 (5 branches checked)

The model was configured with $|\texttt{MaxPolicyVersion}| = 5$, $|\texttt{Actions}| = 3$ (safe_dosage, unsafe_dosage, escalate_case). The invariant I_g held under all 324 distinct states: non-compliant actions were unreachable in the verified bounded model within this configuration. The collision probability across the explored state space is 2.5×10^{-14}.

The TLA+ specification and TLC output log are provided as supplementary artifacts.[1]

TLA+ Prefix Alignment Verification. To formally verify that mid-inference double-buffered DFA updates do not introduce undefined states or execution panics (Optimization A, Section IV-C), the TLA+ specification was extended with the `DfaState` variable and a `PrefixAligned` invariant. TLC verified that under all interleavings of background compilation and active token-level swaps, the active DFA pointer $\mathcal{A}_{\text{active}}$ is

[1] https://github.com/riddhimohansharma/ehv-runtime

only updated when $q_t \in \mathcal{A}_{\Pi'}$. This guarantees that policy hot-swaps maintain strict syntactic prefix alignment, preventing state-space out-of-bounds transitions across the entire concurrent lifecycle.

D. Scope and Small-Model Considerations

The model checking results verify the enforcement logic for a bounded configuration with a single agent variable. Per the small-scope hypothesis [17], most design errors in concurrent and distributed systems manifest in models with small parameter values. We acknowledge the following scope boundaries:

- The specification does not model concurrent multi-agent actions.
- Realistic CRDT merge conflict scenarios with vector clocks are not modeled.
- Unbounded policy version sequences are not covered.

Extension to unbounded state spaces via inductive invariants using the TLA+ Proof System (TLAPS) is identified as primary future work. TLAPS would enable construction of unbounded inductive safety proofs via manual proof obligations; this requires significant formal methods expertise beyond model checking and is a multi-month research investment.

VI. Threat Model

We enumerate the attack surface specific to governance-compiled agentic systems.

A. Trust Assumptions

1) The TEE hardware root of trust is uncompromised (Intel TDX, AMD SEV-SNP at current patch levels).
2) Policy updates are cryptographically signed by authorized issuers.
3) The PEP binary within the TEE is measured and attested at epoch boundaries.
4) Network partitions are eventually resolved (partial synchrony model).
5) Grammar specifications correctly capture the policy intent they represent (grammar correctness is a trust dependency, not a formal guarantee).

B. Threat Categories

TABLE I
EHV Threat Model v2.0

ID	Threat	Mitigation	Residual
T1	Policy poisoning via CRDT injection	Authenticated updates; issuer allowlist; signed DAG mutations; replay protection	Low
T2	TEE side-channel (Spectre/Foreshadow/StackWarp)	LFENCE barriers; SMT disabled; small enclave TCB; firmware patch cadence; epoch rotation	Medium
T3	Stale epoch exploitation	Configurable $\|E_k\|$; forced re-attestation on critical updates; bounded epoch TTL	Low
T4	Network partition $>$ epoch	CRDT convergence; fail-closed halt state; operator escalation	Low
T5	Output obfuscation via adversarial encoding	GCD enforces syntactic grammar constraints; typed action envelopes; note: semantic bypass via syntactically valid payload remains residual	Medium
T6	Human override abuse	Signed approval envelopes; RBAC on escalation paths; GBOM logging	Medium
T7	Clock skew / NTP poisoning	Vector clock ordering; signed causal DAG; physical clock dependency removed	Low

T2 Note (StackWarp, 2026): A newly disclosed architectural vulnerability class (StackWarp, affecting AMD Zen 1 through Zen 5 processors) can manipulate the stack pointer inside an SEV-SNP guest and break guest integrity [33]. Mitigation requires firmware-level patches in addition to the LFENCE and SMT-disable mitigations. EHV deployments on AMD SEV-SNP must maintain a current firmware patch schedule as a non-optional security control.

T2 Note (Performance Impact): The performance impact of host-side TEE mitigations (LFENCE barriers, SMT disable) applies to CPU-bound operations. The GCD logit masking step—the latency-critical enforcement operation—executes as a vectorized array operation on the GPU via the LogitsProcessor interface, outside the CPU mitigation boundary. LLM inference latency is dominated by the GPU forward pass (10–30ms per token); the GCD mask application adds ∼0.033ms per step. However, logit buffers must transit between host CPU memory and GPU memory via PCIe DMA transfers; if host-side LFENCE serialization stalls amplify the synchronization latency between token generation steps, the effective per-step overhead may increase. EHV mitigates this PCIe DMA bottleneck via *Asynchronous Mask Pre-fetching*. Rather than waiting for the raw logits to be generated on the GPU, transferred to the CPU TEE, masked, and returned, the CPU TEE pre-calculates the valid token transition mask Σ_{t+1} based on predicted DFA state transitions and pushes it to a secure GPU memory buffer *in parallel* with the GPU's current token

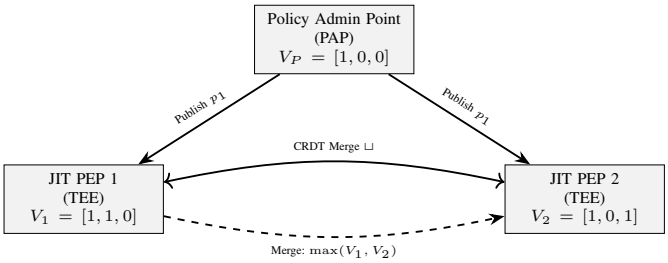


Fig. 3. Causal Graph CRDT Policy Synchronization. Policy updates are propagated across the distributed network using Vector Clocks (Lamport causal ordering) and merged via a Join-Semilattice $\sqcup$, removing dependencies on vulnerable physical clocks (NTP).

forward pass. This hides the PCIe transfer and LFENCE serialization penalties entirely within the GPU compute latency envelope, maintaining sub-millisecond enforcement without pipeline stalls. In practice, PCIe DMA transfers are bus-bandwidth-bound rather than CPU-pipeline-bound, limiting this amplification effect. Empirical measurement of the combined TEE mitigation + DMA synchronization overhead on production hardware is required (Section IX).

T5 Note (GCD Scope): GCD enforces *syntactic* compliance. An adversary who constructs a syntactically valid but semantically harmful action (e.g., a valid dosage token within permitted range that achieves harm through drug interaction context) is not blocked by GCD alone. Defense in depth via human clinical oversight (ESCALATE path) and semantic intent analysis is required for high-stakes action categories.

C. Failure Mode: Non-TEE Environments

In environments lacking Confidential Computing support, the Sub-millisecond enforcement target degrades. The system falls back to out-of-band audit with $GL > 0$, governed by NIST SP 800-53 SI-17 fail-safe provisions [14]. This is the primary architectural friction point for legacy infrastructure deployments.

D. Epoch Staleness Window (ESW) Analysis

Within a policy epoch E_k, a critical policy update arriving at time t_u is not enforced until the next epoch boundary. The maximum staleness window is:

$$ESW_{\max} = |E_k| - 1 \tag{13}$$

For healthcare parameters ($\lambda = 500{,}000$ actions/hour aggregate, $|E_k| = 60$s):

$$N_{\text{stale}} \leq \lambda \cdot ESW_{\max} = \frac{500{,}000}{3{,}600} \times 59 \approx 8{,}194 \text{ actions} \tag{14}$$

Compared to the legacy $N_{\text{unsafe}} = 168{,}000{,}000$ under 14-day GL, this represents a **5 order-of-magnitude reduction**. For ultra-critical updates (e.g., drug withdrawal signals), EHV supports forced mid-epoch re-attestation via an `EMERGENCY_EPOCH_RESET` signal, reducing ESW to network propagation latency (< 1s). The emergency reset channel itself introduces an attack surface: an adversary who can trigger spurious resets creates a Denial of Service (DoS) via continuous re-attestation overhead, while an adversary who can block the signal preserves the staleness window. EHV mitigates reset-channel abuse through cryptographic authentication of reset signals (requiring PAP-level signing authority) and rate-limiting reset acceptance to at most one forced reset per $|E_k|/2$ interval.

E. Fail-Closed Partition Semantics

When a network partition persists beyond the epoch boundary $|E_k|$ and the local node cannot reach the attestation authority, EHV enforces strict fail-closed semantics:

$$t - t_{\text{last_attest}} > |E_k| \implies G(a, C) = \texttt{DENY} \quad \forall a \tag{15}$$

The JIT PEP transitions to a `Safe Halt State`, blocking all outgoing tool and action executions until attestation is restored. This guarantees that no agent executes under an indefinitely stale policy state during extended network outages.

Availability Attack Surface. The fail-closed design creates an adversary-exploitable availability vulnerability: an external attacker who can sustain a network partition (e.g., via targeted network jamming or DNS poisoning against the attestation authority) can force all agents in the affected zone into continuous halt states, crippling operational availability. For safety-critical deployments (e.g., medical monitoring, energy grid control), this trade-off is explicit: EHV prioritizes safety (no action under stale policy) over availability (continuous operation under uncertainty). Deployments requiring high availability under adversarial network conditions should implement redundant attestation authority paths and geographic failover for the attestation endpoint, reducing the single-path partition vulnerability.

VII. Case Study: Pediatric Oncology Dosage

Consider an FDA-mandated reduction in Vincristine dosage from 1.5 mg/m^2 to 0.75 mg/m^2 due to a new neurotoxicity signal.

A. Legacy System (GL = 14 days)

Manual protocol review, committee approval, EHR update, and staff retraining yield $GL = 14$ days. During this interval, across 5,000 Physician Twin instances processing 100 recommendations/hour:

$$N_{\text{toxic}} = 5{,}000 \times 100 \times 24 \times 14 = 168{,}000{,}000 \text{ actions under stale policy} \tag{16}$$

Even at a conservative 0.03% violation rate, this produces 50,400 potentially toxic dosage recommendations before the policy is enforced.

B. EHV System (GL < 1ms, target)

The dosage update propagates via Causal CRDT in <1 second. The compiled GCD grammar for the Vincristine dosage policy sets the maximum permitted token to `0.75`; within the scope of the compiled grammar, any model output exceeding this value is excluded from the token probability mass before sampling (the logit is set to $-\infty$, yielding zero probability after softmax). The action cannot be emitted regardless of the underlying LLM's training distribution. This guarantee is syntactic: it holds for constraints expressible in the policy grammar but does not extend to semantic bypass vectors outside the grammar's scope (Section IV-C).

The GBOM (Section VIII-B) issues a cryptographic receipt binding each clinical recommendation to the specific policy version (Merkle root H_{p}), TEE attestation epoch, and enforcement outcome, enabling post-hoc verification of any recommendation's compliance provenance.

VIII. Discussion

A. The Velocity-Ethics Co-Production Principle

In EHV-compliant systems, deployment velocity V and governance integrity I are positively correlated:

$$\frac{\partial V}{\partial I} \geq 0 \quad \text{(EHV)} \quad \text{vs.} \quad \frac{\partial V}{\partial I} < 0 \quad \text{(traditional)} \tag{17}$$

The causal mechanism underlying this sign reversal is *pre-clearance elimination of post-hoc audit*. In traditional architectures, governance imposes friction as a linear cost: $V = V_0 - k \cdot I$, where k represents audit overhead per unit of integrity. In EHV, because compliance is verified at inference time, no post-deployment audit backlog accumulates. Formally: $V_{\mathrm{EHV}} = V_0 \cdot f(I)$ where $f(I) \geq 1$ when pre-execution verification reduces the need for retrospective compliance gates. Within the EHV enforcement model, governance becomes the mechanism of acceleration rather than friction. This co-production property is conditional on the assumptions in Section VI: TEE integrity, grammar correctness, and eventual CRDT convergence. If any assumption is violated, the system fails closed rather than producing a false co-production signal.

B. Governance Bill of Materials (GBOM) for M&A Due Diligence

EHV introduces the **Governance Bill of Materials (GBOM)**: a cryptographic audit trail that binds each autonomous decision to the specific policy version (Merkle root H_{p}), TEE attestation epoch, GCD DFA state, and enforcement outcome that governed it. The GBOM is formatted as OSCAL Assessment Results (v1.1.2) [31], [32], the NIST-developed schema used in FedRAMP compliance automation. This enables acquirers in M&A due diligence to verify the governance posture of an AI stack with the same rigor applied to financial audits. See Appendix C for a representative OSCAL GBOM sample.

C. Regulatory Alignment

Table II maps EHV controls to applicable regulatory and standards requirements. EHV *supports compliance evidence generation* for these frameworks; it does not replace the organizational controls, legal review, or conformity assessment required for legal compliance.

TABLE II
Controls-to-Requirements Traceability Matrix

Requirement	EHV Control	Evidence Artifact
NIST AI RMF Govern 1.2 (Accountability)	Signed policy lifecycle; audit trail; incident hooks	GBOM; policy mutation log
NIST AI RMF Manage 2.4 (Monitoring)	GCD enforcement + epoch-scoped continuous attestation	Per-action OSCAL observation records
NIST SP 800-207 §3 (PEP/PE/PA)	Identity + per-action verification; SPIFFE SVID credentials; JIT PEP maps to ZTA PEP	Attestation log; token exchange records
NIST SP 800-53 AC-5 (Separation of Duties)	PAP/PDP/PEP separation across trust boundaries (Section IV-F)	Signing key isolation; GBOM independence
EU AI Act Article 12 (Logging)	Per-action signed records; policy version binding	OSCAL assessment-results
FDA PCCP [13]	Constraint-set updateable without redeployment via epoch mechanism	Policy epoch transition log
Human oversight	ESCALATE state with signed approval envelope	Signed override record; GBOM entry
Change control	Epoch update discipline; Merkle-rooted policy commits	Policy mutation log

D. Deployment Topology and Operational Considerations

EHV is designed for *high-stakes regulated verticals*—healthcare clinical agents, financial compliance automation, critical infrastructure control—where the cost of a single unsafe autonomous action exceeds the infrastructure premium of Confidential Computing. It is not positioned as a general-purpose middleware for consumer-facing chatbots or advisory systems.

Confidential Computing infrastructure is available today on all three major clouds: AWS (`c6a.metal` with SEV-SNP), GCP (`C3D` with SEV-SNP), and Azure (`DCasv5` with SEV-SNP/TDX). The cost premium is 10–20% over standard instances for equivalent compute. SPIRE federation enables regional TEE clusters with automatic credential rotation, reducing global synchronization complexity to a SPIRE server mesh operating alongside the CRDT policy store.

E. TEE Portability

The EHV software architecture—GCD engine, CRDT policy store, SPIFFE identity layer, GBOM logging—is specified independently of any TEE vendor. The TEE provides two abstract interfaces: (1) encrypted memory isolation and (2) remote attestation. These interfaces are implemented by AMD SEV-SNP, Intel TDX, ARM CCA, and NVIDIA H100 Confidential Computing. EHV deployments can migrate between TEE backends without modifying the policy compilation, enforcement logic, or audit format.

EHV does require *some* Confidential Computing substrate—it is not hardware-agnostic in the sense of running on commodity hosts. The architecture is TEE-vendor-portable, not TEE-optional. Non-TEE deployments degrade to out-of-band auditing (Limitation 1 below).

F. Comparison: Prompt-Layer Policy-as-Code

An alternative to EHV's approach is Declarative Policy-as-Code (PaC) enforced via system-prompt engineering, with compliance verified through transparency ledgers. Table III compares these architectures.

TABLE III
ARCHITECTURAL COMPARISON: PAC VS. EHV

Property	**System-Prompt PaC + Ledger**	**EHV (GCD + TEE)**
Jailbreak resistance	Low—system prompts are bypassable via adversarial inputs	High—DFA rejects invalid tokens before sampling
Enforcement timing	Post-hoc (logged after execution)	Pre-execution (blocked at token level)
Security guarantee	Probabilistic (LLM may ignore prompt)	Deterministic syntactic constraint (within grammar scope)
Semantic bypass	Vulnerable	Equally vulnerable
Hardware cost	None	10–20% CVM premium
Deployment complexity	Low	High

PaC is appropriate for low-stakes advisory systems where post-hoc accountability suffices. For regulated domains where a single unconstrained action carries regulatory liability (e.g., clinical dosage, financial trade execution), the pre-execution guarantee of GCD+TEE justifies the infrastructure premium.

G. Limitations and Future Empirical Work

1) **TEE dependency**: Confidential Computing hardware is required for the full enforcement model. Non-TEE deployments degrade to out-of-band auditing.
2) **Epoch granularity**: The trade-off between freshness and attestation cost requires domain-specific tuning.
3) **Grammar correctness**: The GCD trust boundary shifts from ASEL parsing fidelity to grammar specification correctness. Formal verification of grammar completeness is future work.
4) **Bounded model checking**: The TLA+ verification covers a bounded configuration (depth 8, 324 distinct states). Extension to unbounded state spaces via TLAPS inductive invariants is primary future work.
5) **No empirical latency data**: The $<$ 1ms enforcement latency target is architectural. Empirical validation on production SEV-SNP hardware is required before this claim can be substantiated in a peer-reviewed venue.
6) **Intra-TEE TCB breadth**: The JIT PEP consolidates CRDT merging, DFA compilation, and logit masking within a single TEE process boundary. Decomposition into isolated micro-enclaves is future hardening work.
7) **Grammar complexity scaling**: DFA state explosion under large-cardinality policy sets within TEE encrypted memory (where paging incurs cryptographic overhead) may introduce performance cliffs. Hierarchical grammar decomposition and empirical memory profiling are required.
8) **Fail-closed availability**: The strict CP design creates an adversary-exploitable availability attack surface via sustained network partitioning. Redundant attestation authority paths mitigate but do not eliminate this risk.

IX. EMPIRICAL VALIDATION PLAN

The following benchmark plan defines the measurement protocol for validating EHV's latency claims. These benchmarks constitute primary future work.

Target Platform: AMD SEV-SNP (AWS, GCP, or Azure confidential VMs). AMD SEV-SNP is selected as the primary MVP target for lower cloud setup friction and broader instance availability relative to Intel TDX [11].

Workload Matrix:

- W1: Safe action permitted (PERMIT baseline).
- W2: Unsafe dosage DENY (GCD masking active).
- W3: Ambiguous clinical case ESCALATE.
- W4: Mid-epoch policy update (boundary behavior).
- W5: Network partition past epoch (fail-closed transition).

Metrics: Per-token latency (baseline vs. GCD vs. GCD+attestation); enclave enter/exit overhead; policy propagation latency; fail-closed transition time.

Reproducibility: Fixed random seed; pinned model version (1B–7B parameter range); frozen policy corpus; recorded instance type, kernel version, SEV-SNP firmware revision, enclave measurement, and policy Merkle root in every row. Hyperthreading disabled to reduce side-channel noise.

Expected overhead based on published SEV-SNP literature: 2–10% for general workloads; <20% for LLM inference; remote attestation cost amortized to < 1 ms per inference via epoch caching [11].

X. Conclusion

EHV transforms governance from a manual gate into a hardware-rooted system invariant. By elevating Grammar-Constrained Decoding to the primary Policy Enforcement Point within the token-generation pipeline, backed by Causal CRDT-synchronized policy state and TEE-anchored attestation caching, EHV reduces Governance Latency by orders of magnitude for agentic systems in regulated domains.

The bounded formal verification demonstrates that non-compliant actions were unreachable in the verified bounded state space under all explored interleavings. The Velocity-Ethics Co-Production Principle establishes that governance integrity and deployment speed are not adversarial but co-productive when enforcement is architectural rather than procedural.

EHV's differentiating contribution is the integration of four independently validated mechanisms—GCD, Causal CRDT, TEE attestation caching, and bounded formal verification—into a single, hardware-rooted enforcement architecture for regulated agentic systems. No contemporaneous system achieves this four-way integration.

Future work includes:

1) Extending the TLA+ specification to unbounded state spaces via TLAPS inductive invariants.
2) Benchmarking enforcement latency on production TEE hardware (AMD SEV-SNP, Intel TDX) per the plan in Section IX.
3) Formal grammar completeness verification for healthcare-specific clinical action schemas.
4) CRDT propagation latency characterization under realistic WAN conditions.

AI Tool Disclosure

AI tools (Claude, Gemini) were used for prose refinement, LaTeX formatting, and literature survey assistance. All technical claims, formal specifications (TLA+), architectural design decisions, mathematical derivations, and the system architecture are the original intellectual work of the author. The TLA+ specification was authored and independently verified by the researcher using the TLC model checker. The proof-of-concept implementation was developed by the author.

Acknowledgments

The author acknowledges the NIST CAISI working group whose publications informed the identity model in Section IV-E.

Appendix

A. DFA Compilation

Given a policy Π expressed as a Context-Free Grammar (CFG) over the action vocabulary, we compile Π to a DFA $\mathcal{A}_\Pi$ using standard subset-construction. For efficiency, we represent $\mathcal{A}_\Pi$ as a Compressed Sparse Row (CSR) matrix over the token vocabulary, enabling $\mathcal{O}(1)$ transition lookup per token.

B. Logits Masking Pseudocode

Require: prefix tokens τ, raw logits L_t, DFA $\mathcal{A}_\Pi$
  $q \leftarrow \mathcal{A}_\Pi$.start
  for $t \in \tau$ **do**
    $q \leftarrow \mathcal{A}_\Pi$.transition$(q, t)$
    if $q = \emptyset$ **then**
      return DENY
    end if
  end for
  $\Sigma_t \leftarrow \mathcal{A}_\Pi$.allowed$(q)$
  for $k \in$ Vocabulary **do**
    if $k \notin \Sigma_t$ **then**
      $L_t[k] \leftarrow -\infty$
    end if
  end for
  return L_t

C. Integration Rules

- Masking MUST occur before softmax at every generation step.
- The DFA state q_t and policy Merkle root H_p are logged to the GBOM for each token.
- Action outputs are wrapped in typed envelopes before downstream tool execution.
- The GCD engine runs within encrypted guest memory; the DFA state is sealed by the TEE.

The following is a representative OSCAL 1.1.2 Assessment Results record binding a single clinical action to its governance provenance:


```
{
  "$schema":
    "https://pages.nist.gov/OSCAL/schemas/
     json/1.1.2/oscal_assessment-results_schema.json",
  "assessment-results": {
    "uuid": "f81d4fae-7dec-11d0-a765-00a0c91e6bf6",
    "metadata": {
      "title": "EHV Runtime Governance Bill of Materials",
      "last-modified": "2026-05-27T00:00:00Z",
      "version": "2.0.0",
      "oscal-version": "1.1.2"
    },
    "results": [
      {
        "uuid": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "title": "Dosage Recommendation Action - Epoch E-042",
        "description": "Vincristine dosage recommendation",
        "start": "2026-05-27T00:00:00Z",
        "end":   "2026-05-27T00:00:00.001Z",
        "reviewed-controls": {
          "control-selections": [
            {"include-controls": [{"control-id": "si-17"}]}
          ]
        },
        "observations": [
          {
            "uuid": "obs-001",
            "description": "GCD enforcement outcome",
            "props": [
              {"name": "policy_merkle_root",
               "value": "sha256:abc123..."},
              {"name": "tee_measurement",
               "value": "sevsnp:mrenclave:def456..."},
              {"name": "epoch_id",      "value": "E-042"},
              {"name": "dfa_state",     "value": "q17"},
              {"name": "enforcement",   "value": "PERMIT"},
              {"name": "spiffe_svid",
               "value": "spiffe://ehv.example/agent/twin-001"}
            ]
          }
        ],
        "findings": [
          {
```

```
            "uuid": "find-001",
            "title": "Action compliant with epoch policy",
            "description":
              "Dosage within GCD-enforced range 0.0-0.75 mg/m2",
            "target": {
              "type": "objective-id",
              "target-id": "ehv-si-17",
              "status": {"state": "satisfied"}
            }
          }
        ]
      }
    ]
  }
}
```


REFERENCES

[1] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023. https://www.nist.gov/itl/ai-risk-management-framework

[2] International Organization for Standardization, "ISO/IEC 42001:2023 — Artificial intelligence — Management system," 2023. https://www.iso.org/standard/81230.html

[3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST SP 800-207, Aug. 2020. https://doi.org/10.6028/NIST.SP.800-207

[4] European Parliament, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)," June 2024. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[5] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*, Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html

[6] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978. https://doi.org/10.1145/359545.359563

[7] C. Newcombe, T. Rath, F. Zhang, B. Murat, and M. Brooker, "How Amazon Web Services Uses Formal Methods," *Commun. ACM*, vol. 58, no. 4, pp. 66–73, 2015. https://doi.org/10.1145/2699417

[8] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete Problems in AI Safety," arXiv:1606.06565, 2016. https://arxiv.org/abs/1606.06565

[9] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, 2022. https://arxiv.org/abs/2212.08073

[10] V. Costan and S. Devadas, "Intel SGX Explained," *IACR Cryptology ePrint Archive*, 2016. https://eprint.iacr.org/2016/086

[11] AMD, "AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More," AMD White Paper, 2020. https://www.amd.com/system/files/TechDocs/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf

[12] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free Replicated Data Types," in *Proc. SSS 2011*, LNCS, vol. 6976, pp. 386–400, 2011. https://doi.org/10.1007/978-3-642-24550-3_29

[13] U.S. Food and Drug Administration, "Marketing Submission Recommendations for a Predetermined Change Control Plan for AI/ML-Enabled Device Software Functions," FDA Guidance, 2023. https://www.fda.gov/regulatory-information/search-fda-guidance-documents/marketing-submission-recommendations-predetermined-change-control-plan-artificial-intelligence

[14] Joint Task Force, "Security and Privacy Controls for Information Systems and Organizations," NIST SP 800-53 Rev. 5, Sept. 2020. https://doi.org/10.6028/NIST.SP.800-53r5

[15] NVIDIA, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications," NVIDIA Technical Report, 2023. https://github.com/NVIDIA/NeMo-Guardrails

[16] Guardrails AI, "Guardrails: Adding Guardrails to Large Language Models," GitHub Repository and Documentation, 2023. https://github.com/guardrails-ai/guardrails

[17] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, MIT Press, 2006.

[18] T. Dong et al., "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models," arXiv:2411.15100, 2024. https://arxiv.org/abs/2411.15100

[19] R. Willard et al., "Efficient Guided Generation for Large Language Models," arXiv:2307.09702, 2023. https://arxiv.org/abs/2307.09702

[20] Cloud Native Computing Foundation, "SPIFFE: Secure Production Identity Framework For Everyone," CNCF Graduated Project, 2022. https://spiffe.io

[21] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Liu, "OAuth 2.0 Token Exchange," RFC 8693, Internet Engineering Task Force, Jan. 2020. https://www.rfc-editor.org/rfc/rfc8693

[22] National Institute of Standards and Technology, "Announcing the AI Agent Standards Initiative for Interoperable and Secure Innovation," NIST Announcement, Feb. 2026. https://www.nist.gov/node/1906621

[23] NIST National Cybersecurity Center of Excellence, "Accelerating the Adoption of Software and AI Agent Identity and Authorization," NCCoE Concept Paper (Public Comment Closed Apr. 2026), 2026. https://www.nccoe.nist.gov/projects/software-and-ai-agent-identity-and-authorization

[24] Linux Foundation, "Linux Foundation Announces the Formation of the Agentic AI Foundation (AAIF), Anchored by New Project Contributions Including Model Context Protocol (MCP), goose and AGENTS.md," Dec. 2025. https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation

[25] C. L. Wang, T. Singhal, A. Kelkar, and J. Tuo, "MI9: An Integrated Runtime Governance Framework for Agentic AI," arXiv:2508.03858, 2025. https://arxiv.org/abs/2508.03858

[26] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," arXiv:2503.18666, 2025. https://arxiv.org/abs/2503.18666

[27] A. M. Mazzocchetti, "Cryptographic Runtime Governance for Autonomous AI Systems: The Aegis Architecture for Verifiable Policy Enforcement," arXiv:2603.16938, 2026. https://arxiv.org/abs/2603.16938

[28] J. Namaya, "SAFi: Self-Alignment Framework Interface," GitHub Repository, 2026. https://github.com/jnamaya/SAFi

[29] S. Abdollahi, M. M. Maheri, J. Forough, A. Al Sadi, J. Millar, D. Kotz, M. Kogias, and H. Haddadi, "AgenTEE: Confidential LLM Agent Execution on Edge Devices," in *Proc. EuroMLSys 2026*, 2026. https://doi.org/10.1145/3805621.3807660

[30] R. Han, Z. Yang, C. Ma, Y. Liu, Y. Niu, S. Ma, D. Gao, and D. Lo, "AutoTEE: Automated Migration and Protection of Programs in Trusted Execution Environments," arXiv:2502.13379, 2025. https://arxiv.org/abs/2502.13379

[31] National Institute of Standards and Technology, "OSCAL Assessment Results Model v1.1.2 JSON Format Reference," 2025. https://pages.nist.gov/OSCAL-Reference/models/v1.1.2/assessment-results/json-reference/

[32] R. Cilla Ugarte, M. A. Patricio Guisado, A. Berlanga de Jesús, and J. M. Molina López, "Making AI Compliance Evidence Machine-Readable," arXiv:2604.13767, Apr. 2026. https://arxiv.org/abs/2604.13767

[33] R. Zhang, T. Hornetz, D. Weber, F. Thomas, and M. Schwarz, "StackWarp: Breaking AMD SEV-SNP Integrity via Deterministic Stack-Pointer Manipulation through the CPU's Stack Engine," in *Proc. USENIX Security*, 2026. https://www.usenix.org/conference/usenixsecurity26/presentation/zhang-ruiyi